



DEPARTAMENTO DE ELECTRÓNICA  
2<sup>do</sup> Cuatrimestre de 2025

---

**TB075/86.53 Procesamiento del Habla**  
**TPfinal**

---

BRIAN FUENTES

101785

# Índice

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Objetivo</b>                                                          | <b>3</b>  |
| <b>2</b> | <b>Desarrollo</b>                                                        | <b>3</b>  |
| 2.1      | Reconocedor de habla continua . . . . .                                  | 3         |
| 2.2      | Preparación de datos y directorios . . . . .                             | 4         |
| 2.3      | Parametrización de los datos . . . . .                                   | 7         |
| 2.4      | Entrenamiento . . . . .                                                  | 10        |
| 2.4.1    | Creación del diccionario . . . . .                                       | 12        |
| 2.4.2    | Creación de MLF de transcripciones de palabra y fonéticas . . . . .      | 14        |
| 2.4.3    | Creación de modelos de monofonos con una sola gaussiana . . . . .        | 15        |
| 2.4.4    | Reconocimiento con el modelo de unigramas y una sola gaussiana . . . . . | 20        |
| 2.4.5    | Refinamiento del modelo agregando mezclas de gaussianas . . . . .        | 24        |
| 2.5      | Implementación de la gramática finita. . . . .                           | 28        |
| 2.5.1    | Evaluación . . . . .                                                     | 34        |
| <b>3</b> | <b>Conclusiones</b>                                                      | <b>36</b> |
| <b>4</b> | <b>ANEXO</b>                                                             | <b>37</b> |

# 1. Objetivo

## Objetivos del proyecto

- implementar un sistema de reconocimiento automático de habla en idioma español de aproximadamente 3000 palabras.
- Implementar un sistema de reconocimiento de gramática finita de alrededor de 30 palabras para ser usado en tiempo real.
- Se implementará un baseline con la base de datos latino40 con modelos de monofonos. Luego se implementará una gramática finita para usarla en tiempo real.

# 2. Desarrollo

## 2.1. Reconocedor de habla continua

Recordando el principio en el que teníamos un modelo donde se representan todas las palabras del vocabulario, y contamos con secuencias de observación en las que aparece una de esas palabras.

Tomamos la secuencia de observación, la convertimos en cepstrum para poder reconocerla, y se la damos al modelo. Luego, por ejemplo, usamos el likelihood para la primera secuencia y para la segunda, y así decidimos qué palabra es mediante un argmax. tal como se muestra en 2.1.1

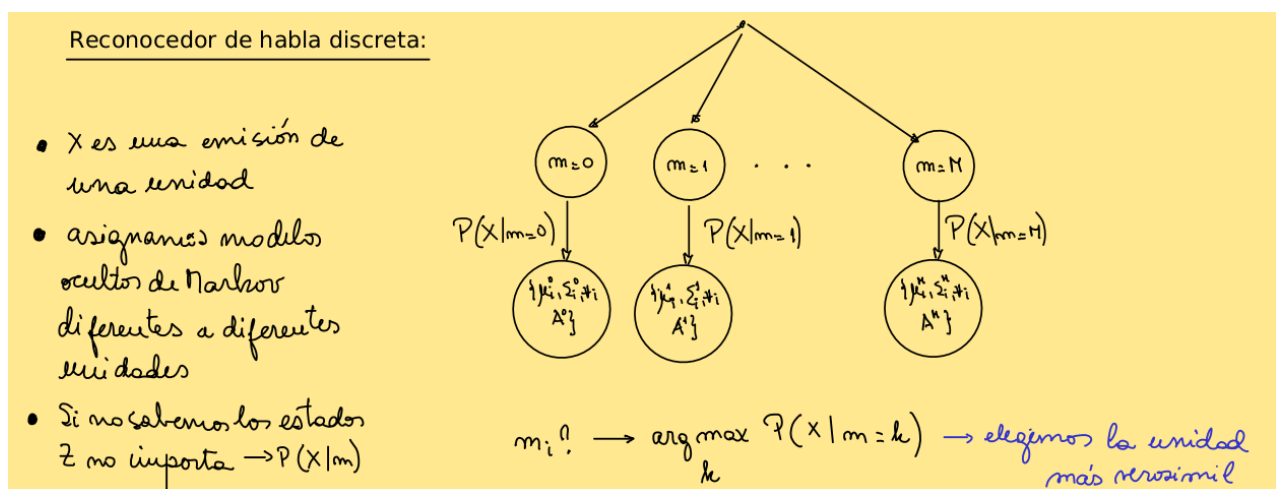


Figura 2.1.1: Modelo discreto

Pero ese no es el esquema que se usa en este caso por ser muy impráctico, porque se necesita que la secuencia  $X$  contenga frames de cada una de las palabras, y eso es muy difícil porque uno habla de forma continua. Lo que se hace es lo siguiente 2.1.2:

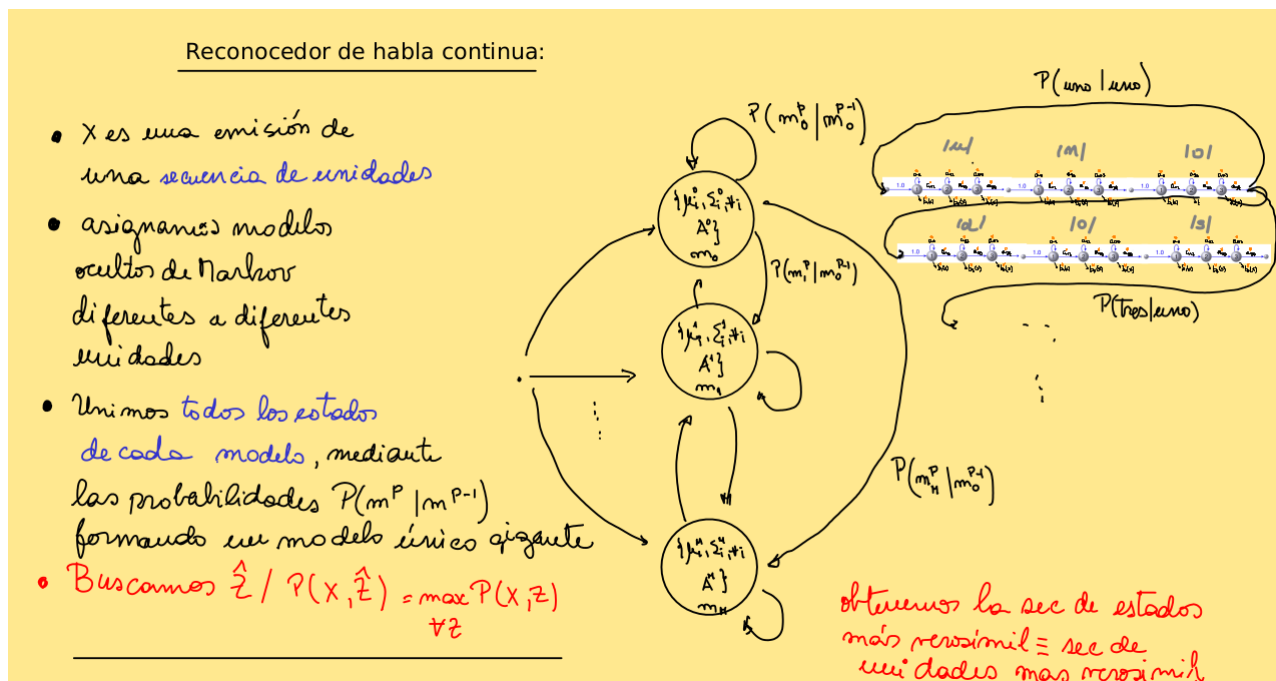


Figura 2.1.2: habla continua

## 2.2. Preparación de datos y directorios

Se conectan los modelos de palabras con modelos de transición sacados de los modelos de lenguaje. Ahora habrá que definir cuál va a ser la unidad, y estas podrían ser las palabras; sin embargo, las palabras pueden ser muchas y muy difíciles de contemplarlas todas debido a que se tienen pocas muestras en el set de entrenamiento, incluso muy difíciles por sí mismas.

Lo mejor no va a ser modelar palabras sino subunidades, y con esas subunidades formar todas las palabras que se nos ocurran.

Esto tiene la ventaja de que tendríamos muchos ejemplos para entrenamiento, aunque tiene la desventaja de que, dependiendo del vocabulario, haya fonemas más representados que otros.

Entonces, para entrenar vamos a tener que tener todas las subunidades de nuestro vocabulario y entrenar las subunidades.

Se va a tener que conseguir un listado de palabras a fonemas; eso define nuestras palabras a tener. También las probabilidades de saltar de un modelo a otro, el modelo de lenguaje  $P(m_H^p | m_h^{p-1})$

Vamos a crear nuestro directorio `proyecto_hc` y su estructura.

### Crear los directorios principales

```
mkdir proyecto_hc
cd proyecto_hc/
mkdir config datos etc lm log modelos rec scripts
```

### Copiar los archivos requeridos

Los archivos requeridos a copiar van a estar en la carpeta `/home/cestien/Documentos/`

```
cp /home/cestien/Documentos/config.hcopy ./config/
cp /home/cestien/Documentos/go.mfclist ./scripts/
```

### Crear subdirectorios dentro de datos

```
mkdir datos/mfc
mkdir datos/wav
```

### Visualizar el árbol de carpetas

```
tree -L 2
```

```
.
|-- config
|   '-- config.hcopy
|-- datos
|   |-- mfc
|   '-- wav
|-- etc
|-- lm
|-- log
|-- modelos
|-- rec
'-- scripts
    '-- go.mfclist
```

10 directories, 2 files

Ahora lo que debemos hacer es pasar los archivos de audio a la carpeta `datos/wav` y los archivos de "texto" a la carpeta `datos/mfc`.

### Enlazar los archivos wav desde otro directorio (enlace simbólico)

Los `wav` están en otro directorio, así que vamos a **enlazarlos** (no queremos copiarlos).

Se sabe que tenemos dos directorios `train` y `devel`. queremos que estos sean los subdirectorios `train` y `test` dentro de `datos`, respectivamente. Para ello utilizamos enlaces simbólicos (`-s`) con el comando `ln`.

```
bfuentes@habla: /proyecto_hc/$ ls /dbase/latino40/
devel  doc  leer  train

bfuentes@habla: /proyecto_hc/datos/$ cd wav/
bfuentes@habla: /proyecto_hc/datos/wav/$ ln -s /dbase/latino40/train/ train
bfuentes@habla: /proyecto_hc/datos/wav/$ ln -s /dbase/latino40/devel/ test
bfuentes@habla: /proyecto_hc/datos/wav/$ ls
test  train
```

Se puede verificar que los enlaces simbólicos se crearon correctamente en la carpeta `datos/wav`:

```
bfuentes@habla: /proyecto_hc/datos/wav/$ tree -L 2
.
    test -> /dbase/latino40/devel/
    train -> /dbase/latino40/train/

2 directories, 0 files
```

### Crear las carpetas `train` y `test` en `mfc`

Ahora, se deben crear las carpetas `train` y `test` en `mfc` para que el sistema pueda crear los archivos de `mfc`.

```
bfuentes@habla: /proyecto_hc/datos/mfc/$ mkdir test train
```

Se tiene que tener la siguiente estructura final:

```
.
|-- config
|   '-- config.hcopy
|-- datos
|   |-- mfc
|   |   |-- test
|   |   '-- train
|   '-- wav
|       |-- test -> /dbase/latino40/train/
|       '-- train -> /dbase/latino40/train/
|-- etc
|-- lm
|-- log
|-- modelos
|-- rec
'-- scripts
    '-- go.mfclist

14 directories, 2 files
```

## 2.3. Parametrización de los datos

La codificación puede realizarse utilizando la herramienta **HCOPY**, configurada para convertir automáticamente su entrada en vectores MFCC.

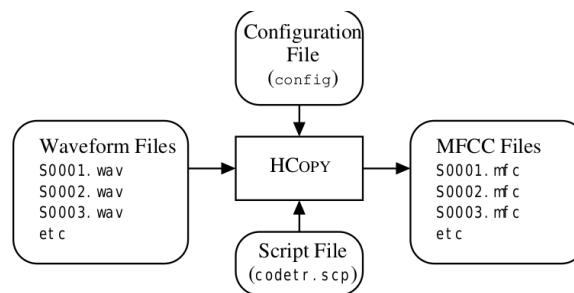


Fig. 3.6 Step 5

Assuming that the above script is stored in the file `codetr.scp`, the training data would be coded by executing

```
HCOPY -T 1 -C config -S codetr.scp
```

**Figura 2.3.1:** HCopy recibe archivos de audio y, utilizando un archivo de configuración, extrae características acústicas, generando como salida archivos de coeficientes cepstrales MFCC (.mfc).

Para ello, se requiere un archivo de configuración (`config.hcopy`) que especifique todos los parámetros de conversión. Valores razonables para dichos parámetros son los siguientes.

```
bfuentes@habla: /proyecto_hc/$ cat config/config.hcopy
# Coding parameters
TARGETKIND = MFCC_0
#TARGETKIND = MFCC_0_D_A
TARGETRATE = 100000.0
SAVECOMPRESSED = T
SAVEWITHCRC = T
WINDOWSIZE = 250000.0
USEHAMMING = T
PREEMCOEF = 0.97
NUMCHANS = 26
CEPLIFTER = 22
NUMCEPS = 12
ENORMALISE = F
SOURCEFORMAT = NIST
```

Estos parámetros corresponden a la configuración del experimento. En este caso, los valores ya han sido determinados previamente; sin embargo, si no se contara con esta configuración, sería necesario realizar pruebas empíricas hasta llegar a un conjunto de parámetros adecuados.

A continuación se describe el significado de los principales parámetros:

- **TARGETKIND:** Define el tipo de coeficientes cepstrales a utilizar. En este caso se

emplea `MFCC_0`, lo que indica que se utilizan coeficientes MFCC incluyendo el coeficiente de energía  $c_0$ .

El coeficiente cepstral  $c_i$  se calcula como:

$$c_i = \sqrt{\frac{2}{N}} \sum_{j=1}^N m_j \cos \left( \frac{\pi i}{N} (j - 0,5) \right),$$

donde

$$m_j = \log(4 \cdot j).$$

En particular, el coeficiente de energía  $c_0$  se obtiene como:

$$c_0 = \sqrt{\frac{2}{N}} \sum_{j=1}^{\text{NUMCHANS}} \log(4 \cdot j).$$

- **TARGETRATE**: Define el periodo de análisis entre ventanas consecutivas, expresado en unidades de 100 ns. Un valor de `100000.0` corresponde a un desplazamiento de 10 ms (2.3.2).
- **WINDOWSIZE**: Tamaño de la ventana de análisis, también en unidades de 100 ns. El valor `250000.0` corresponde a una ventana de 25 ms (2.3.2).
- **USEHAMMING**: Indica si se aplica una ventana de Hamming antes del análisis espectral. Su uso reduce los efectos de discontinuidad en los bordes de la ventana.
- **PREEMCOEF**: Coeficiente de preénfasis aplicado a la señal de entrada. Un valor típico es 0,97, que amplifica las componentes de alta frecuencia.
- **NUMCHANS**: Número de filtros en el banco de filtros Mel. En este caso se utilizan 26 canales.
- **NUMCEPS**: Número de coeficientes cepstrales retenidos. Acá se utilizan 12 coeficientes, excluyendo  $c_0$  del conteo cuando se especifica por separado.
- **CEPLIFTER**: Parámetro de liftering cepstral, que enfatiza coeficientes de orden medio para mejorar la discriminabilidad (es como un ponderado para que los coeficientes sean comparables).
- **ENORMALISE**: Indica si se aplica normalización de energía. En este caso está deshabilitada (F).
- **SOURCEFORMAT**: Especifica el formato de los archivos de audio de entrada. Acá se utiliza el formato NIST.

El archivo `textttgo.mfclist` es un script de *shell* que recorre un directorio de entrada y, a medida que procesa los archivos de audio, va creando automáticamente los subdirectorios correspondientes. En esencia, este script automatiza el proceso de generación de los archivos `mfc` (debe ser copiado de CESTIEN). Para poder ejecutarlo, es necesario agregar permisos de ejecución al archivo `mfc`:



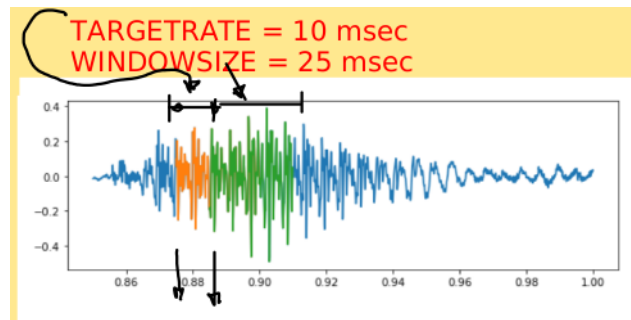


Figura 2.3.2: TARGETRATE, WINDOWSIZE

```
bfuentes@habla: /proyecto_hc/scripts/$ chmod +x go.mfclist
```

El script debe ejecutarse desde la carpeta `datos`, ya que las rutas internas están definidas de forma relativa.

```
bfuentes@habla: /proyecto_hc/datos/$ ../scripts/go.mfclist genmfc.train
genmfc.test
```

Una vez ubicados en dicho directorio, la generación de los coeficientes MFCC se realiza mediante el comando `HCOPY`, ejecutándolo de manera independiente para los conjuntos de entrenamiento y prueba:

```
bfuentes@habla: /proyecto_hc/datos/$ HCopy -T 1 -C ../config/config.hcopy -
S genmfc.train
bfuentes@habla: /proyecto_hc/datos/$ HCopy -T 1 -C ../config/config.hcopy -
S genmfc.test
```

Los parámetros utilizados en el comando `HCOPY` son los siguientes:

- **-T**: Define el nivel de *trace* o nivel de salida informativa durante la ejecución del comando. Valores mayores generan mayor cantidad de mensajes.
- **-C**: Especifica el archivo de configuración, en este caso `config.hcopy`, que contiene los parámetros para la extracción de características.
- **-S**: Indica el archivo de entrada (*script file*), el cual lista los archivos de audio a procesar y sus correspondientes rutas de salida.

Veamos los archivos **de entrada**.

```
bfuentes@habla: ~/proyecto_hc/datos$ HList -F NIST -h -e 10 wav/test/af14/af14_001.wav
----- Source: wav/test/af14/af14_001.wav -----
Sample Bytes: 2      Sample Kind:  WAVEFORM
Num Comps:    1      Sample Period: 62.5 us
Num Samples:  73601  File Format:   NIST
----- Samples: 0->11 -----
0:    452    463    471    470    469    473    478    469    478    495
10:   481    491
----- END -----
```

y los **generados**

```
bfuentes@habla:~/proyecto_hc/datos$ HList -F HTK -h -e 10 mfc/test/af14/af14_001.mf
----- Source: mfc/test/af14/af14_001.mfc -----
Sample Bytes: 26      Sample Kind: MFCC_C_K_0
Num Comps: 13      Sample Period: 10000.0 us
Num Samples: 458    File Format: HTK
----- Samples: 0->10 -----
0:      -8.213 -0.047  4.862  0.893  2.936  3.283 -0.177 -1.851  7.483 -0.
        2.952  6.463 48.605
1:      -8.731 -2.581  3.314  2.989  2.998  4.048  5.614  5.891  4.754 -2.
        5.703  5.149 48.804
2:      -9.706 -2.024  0.659 -3.607  2.956  1.461  0.109  9.757  8.703 -1.
        6.040 12.030 47.996
3:      -8.147 -2.294  1.494  2.075  2.958  1.552 -0.418  1.645  3.176  0.
        6.136  7.873 49.005
4:      -7.814 -2.312  1.205  5.998  1.456  2.991  4.417 -1.118 -0.668 -4.
        4.224  6.895 49.625
5:      -9.705 -3.319  2.350  3.940  3.942  5.455  7.870  1.300  5.112  1.
        4.535  7.733 48.715
6:     -11.344 -4.738  4.436  2.068  3.728  8.468 14.160  7.139  6.756  3.
        -0.507 -1.550 48.941
7:     -10.380 -3.274  6.432 -0.628 -3.162  2.475  5.817  3.068 -0.982  3.
        4.927 12.957 47.935
8:      -8.119 -2.180  4.742 -0.794 -2.293  2.134  3.807  0.542  3.022  1.
        9.045  8.679 49.517
9:      -9.021 -0.903  6.073  0.798 -0.713  5.973  3.929  1.000 -0.350 -7.
        6.331  7.808 49.232
10:     -9.794 -1.216  6.582  0.567 -0.756  1.010  4.377  3.371  2.629  2.
        8.195  8.027 48.595
----- END -----
```

La salida del comando **HList** presenta información descriptiva del archivo de audio **af14\_001.wav** perteneciente al conjunto de prueba. El archivo se encuentra en formato **NIST** y corresponde a una señal de tipo **WAVEFORM**, con muestras de 16 bits (2 bytes por muestra) y una frecuencia de muestreo de 16 kHz, determinada a partir de un periodo de muestreo de  $62,5 \mu s$ . El archivo contiene un total de 73601 muestras. Además, se muestran los valores de amplitud de las primeras muestras de la señal.

Mientras que el archivo generado es de tipo **HTK**

## 2.4. Entrenamiento

Partimos de una cadena de Markov, en donde tenemos los parámetros  $a_{ij}$  los parámetros de la matriz de transición **A**. También se tiene los parámetros de las tres distribuciones  $b_i(x)$  el cual suponemos son gaussianas y van a pasar a ser más adelante unas mezclas de gaussianas.



En otras palabras, el entrenamiento embebido permite estimar los parámetros de los HMM de los fonemas a partir de habla continua sin segmentación previa. Para cada enunciado, se concatenan los HMM según la transcripción fonética y, mediante el algoritmo Baum–Welch, se infiere probabilísticamente la asignación temporal de los estados. De esta forma, los límites de los fonemas se estiman implícitamente y los parámetros se reestiman a partir de estadísticas acumuladas sobre toda la señal.

### 2.4.1. Creación del diccionario

Se copian archivos.

```
bfuentes@habla: /proyecto_hc/etc/$ cp /home/cestien/Documentos/lexicon ./
bfuentes@habla: /proyecto_hc/etc/$ cp /home/cestien/Documentos/leexicon.gf
./
```

Estos archivos son un diccionario de palabras y sus correspondiente pronunciaciones.

```
bfuentes@habla:~/proyecto_hc/etc$ tail lexicon
zulema          s uw l ey m aa
zulemita        s uw l ey m iy t aa
zulu            s uw l uw
zumbaron        s uw m b aa rx ow n
zumo            s uw m ow
zuniga          s uw ny iy gg aa
zurda           s uw rx dd aa
zurdo           s uw rx dd ow
zurita          s uw rx iy t aa
zutano          s uw t aa n ow
```

EL primero es un diccionario general, el cual contiene palabras y sus pronunciaciones.

```
bfuentes@habla:~/proyecto_hc/etc$ tail lexicon.gf
nueve           n w ey bb ey
ocho            ow ch ow
patricia        p aa t rx iy s y aa
pedro           p ey dd rx ow
perez           p ey rx ey s
rodriguez       rr ow dd rx iy gg ey s
seis            s ey y s
siete           s y ey t ey
tres            t rx ey s
uno             uw n ow
```

Este ultimo es otro diccionario de palabras pero que contiene las palabras de la gramatica finita a aplicar 2.5.2

Luego, se crea global.ded un archivo que va a contener el silencio corto (/sp/) estos son los comandos de edición de la instruccion *HDman*.

```
bfuentes@habla: /proyecto_hc/etc/$ echo "AS sp">global.ded
```

Esto agrega un silencio corto al final de todas las entradas del diccionario.

Luego nos descargamos los diccionarios promptsl40 de cestiene, para train y test

```
bfuentes@habla: /proyecto_hc/etc\$ cp /home/cestien/Documentos/prompts140.
train ./
bfuentes@habla: /proyecto_hc/etc\$ cp /home/cestien/Documentos/prompts140.
test ./
```

Estos archivos contienen las transcripciones de los *archivos.wav* por ejemplo:

```
bfuentes@habla:~/proyecto_hc/etc$ tail prompts140.test
vm25_116 esto no significa que no lo haran pero sera dificil
vm25_117 no veo razon alguna para que baje expreso un agente
vm25_118 a menudo se requiere mas de un documento para ello
vm25_119 hemos concluido asi las audiencias sobre el tema
vm25_120 veanse las definiciones correspondientes al cuadro seis
vm25_121 no jugara el resto de la temporada regular
vm25_122 me he repuesto mas rapidamente de lo que pensaba
vm25_123 el tema sigue sometido a un estudio constante
vm25_124 los cambios al marcador fueron hechos por el australiano
vm25_125 monselas nego que le hubieran pagado por el homicidio
```

Ahora la idea es generar una lista de palabras con todo el contenido, sin repeticiones. Lo reaslizamos con **awk** que nos permite operar linea a linea con los archivos. y asi obtener **wlist40** una lista de palabras unicas y ordenadas generadas a partir de los promptsl40

```
bfuentes@habla: /proyecto_hc/etc\$ cat prompts140.train prompts140.test |
awk '{for(n=2; n<=NF; n++)print $n}' | sort | uniq > wlist140
```

```
awk '{for(i=2;i<=NF;i++){print $i}}'|sort|uniq > wlist140
```

sort: ordena las palabras.

uniq: elimina las palabras repetidas.

```
awk:
$1      $2      $3      $4
"primera segunda tercera cuarta"-> $0:primera | segunda | tercera | cuarta ->record
      ^_field
```

NF=: Numero de field en cada record.

NR=: Numero de record.

awk va recorriendo linea por linea y hace la instruccion que yo le pida.

```
bfuentes@habla: /proyecto_hc/etc\$ HDMan -m -w wlist1 -g global.ded -n
monophones+sil -l ../log/hdman.log dictl40 lexicon lexicon.gf
```

Donde **dictl40** será el diccionario de salida,, **lexicon** será el diccionario base y **lexicon.gf** el diccionario de gramatica finita que complementará al anterior. los demás parametros son:

- **-m**: Indica que los diccionarios de entrada se fusionan, gestiona duplicados.

- **-w:** *wlistl* contiene la lista de palabras válidas, *dictl40* contendrá palabras de la lista.
- **-n:** Lista los fonemas encontrados, le indica el archivo de edición global.ded
- **-l:** ruta a donde se guardarán los logs.

Finalmente obtenemos el siguiente diccionario que usaremos de acá en adelante.

```
bfuentes@habla:~/proyecto_hc/etc$ tail dictl40
zona          s ow n aa sp
zonas         s ow n aa s sp
zoologico     s ow ow l ow hh iy k ow sp
zulema        s uw l ey m aa sp
zulemita      s uw l ey m iy t aa sp
zulu          s uw l uw sp
zuniga        s uw ny iy gg aa sp
zurita        s uw rx iy t aa sp
```

#### 2.4.2. Creación de MLF de transcripciones de palabra y fonéticas

La información de transcripciones se proveerá a partir de master label files (MLF). Estos archivos tienen un formato especial en el cual se provee la ubicación del archivo que contiene la emisión acústica y el correspondiente archivo de transcripción.

Si la transcripción es por palabras, en cada línea habrá una palabra; si es por fonema, en cada línea habrá un fonema. Opcionalmente se puede agregar una short-pause (sp) entre palabras y un silencio (sil) al comienzo y al final de cada transcripción.

Para implementar el MLF de palabras puede utilizar el script **script prompts2mlf** provisto en `/home/cestien/proyecto`. La implementación del MLF fonético se realiza con HLEd.

Esta aplicación posee varias posibilidades para editar archivos de etiquetas (transcripciones). Requiere como entrada el MLF a nivel palabras, un archivo con los comandos que implementan las acciones deseadas, y el diccionario.

El archivo con los comandos se llama `mkphones-sp.led` y también está provisto en `/home/cestien/proyecto`. Como resultado produce el archivo MLF a nivel fonético.

Empecemos copiando el script, se le debe de dar permisos de ejecución y se lo procede a ejecutar desde la carpeta `/etc`.

```
bfuentes@habla: /proyecto_hc/script/$ cp /home/cestien/Documentos/
prompts2mlf ./
bfuentes@habla: /proyecto_hc/script/$ chmod +x prompts2mlf
bfuentes@habla: /proyecto_hc/etc/$
../scripts/prompts2mlf mlftrain promtsl40.train
```

Esto genera un master label file a nivel de palabras, *mlftrain*.

```
bfuentes@habla:~/proyecto_hc/etc$ head mlftrain
#!MLF!#
"af01_001.lab"
no
```

```

habiendo
objeciones
asi
quedo
acordado
.
"af01_002.lab"

```

```

bfuentes@habla:~/proyecto_hc$ head etc/mkphones0.led
EX                                (expandir palabras usando diccionario.)
IS sil sil                        (insertar silencios.)
DE sp                             (eliminar pausas cortas.)

```

Este comando usa HLEd (HTK Label Editor) para convertir transcripciones de palabras en transcripciones de fonemas, aplicando reglas definidas en un archivo **.led**.

```

bfuentes@habla: /proyecto_hc/script/$ HLEd -l '*' -d dictl40 -i fonemas0.
mlf mkphones0.led mlftrain

```

- -l '\*': Aplica las reglas de edición a todos los labels.
- -d dictl40: Diccionario fonético (palabras → fonemas).
- -i fonemas0.mlf: Archivo MLF de salida con transcripciones a nivel fonema.
- mkphones0.led: Archivo de reglas que define cómo convertir palabras en fonemas.
- mlftrain: MLF de entrada con transcripciones a nivel palabra.

La salida ya son las transcripciones a nivel fonema sin pausas cortas (sp).

```

bfuentes@habla:~/proyecto_hc$ head etc/fonemas0.mlf
#!MLF!#
"/af01_001.lab"
sil
n
ow
aa
bb
y
ey
n

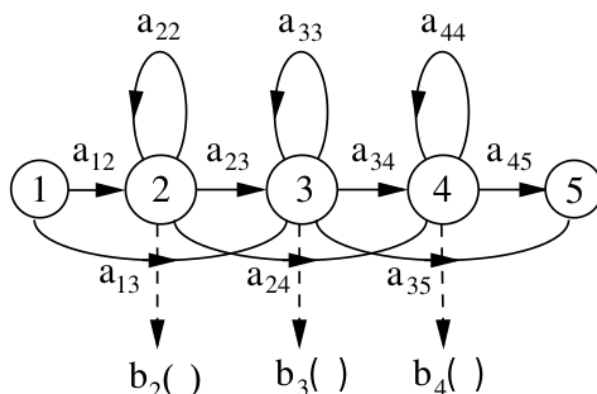
```

### 2.4.3. Creación de modelos de monofonos con una sola gaussiana

Se crean los modelos de fonemas; para cada fonema hay 5 estados, de los cuales el primero y el último serán los de inicio y fin respectivamente.

Los 3 estados del medio serán los estados emisores y sus probabilidades de pasar de un estado a otro darán lugar a una matriz de transición  $P$ ; a su vez, cada uno de estos estados

es modelado por una gaussiana n-dimensional (diagonales y, en este caso, NumMixes=1 no aparece).



**Figura 2.4.3.1:** Modelos HMM de 5 estados-bakis

La descripción de este modelo se implementa en formato model file

Este modelo para un fonema en particular se vé de la siguiente manera.

```

bfuentes@habla:~/proyecto_hc$ head -n 30 modelos/hmm0/hmmdefs
~h "aa"
<BEGINHMM>
<NUMSTATES> 5
<STATE> 2
<MEAN> 39
-7.610806e+00 -4.806952e+00 3.343391e+00 ...
<VARIANCE> 39
4.912927e+01 3.201926e+01 4.591968e+01 ...
<GCONST> 1.254290e+02
<STATE> 3
<MEAN> 39
-7.610806e+00 -4.806952e+00 3.343391e+00 ...
<VARIANCE> 39
4.912927e+01 3.201926e+01 4.591968e+01 ...
<GCONST> 1.254290e+02
<STATE> 4
<MEAN> 39
-7.610806e+00 -4.806952e+00 3.343391e+00 ...
<VARIANCE> 39
4.912927e+01 3.201926e+01 4.591968e+01 6...
<GCONST> 1.254290e+02
<TRANSP> 5
0.000000e+00 1.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00
0.000000e+00 6.000000e-01 4.000000e-01 0.000000e+00 0.000000e+00
0.000000e+00 0.000000e+00 6.000000e-01 4.000000e-01 0.000000e+00
0.000000e+00 0.000000e+00 0.000000e+00 7.000000e-01 3.000000e-01

```



```
0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00 0.000000e+00
<ENDHMM>
```

Donde GCONST es la constante independiente de la observación  $b_j$  al cual se le aplica logatimo y queda ese termino restando.

TRANSP es la matriz de transicion los  $a_{ij}$ , los demás tags son faciles de entender.

Los modelos iniciales se crearán usando HCompV; también se creará con esta herramienta el macro vFloors, que establece un piso para el valor de la varianza en los sucesivos modelos.

Todos los modelos se encuentran en un archivo llamado hmmdefs. La idea es similar a la de los MLF pero para modelos; por ello este tipo de archivos se llaman **master models file**(MMF). Para crear el archivo hmmdefs inicial utilice el script go.gen-hmmdefs. Este requiere como entrada una lista de fonos y el prototipo (proto). La lista de fonos ya la obtuve cuando creó el diccionario usando la opción -n en HDMan. Se utiliza dicha lista, pero se elimina **sp**, ya que **sp** no se modeliza con la topología de proto. Además, se agrega **sil**, ya que no está en la lista.

```
bfuentes@habla: /proyecto_hc/modelos/$ ls ../datos/mfc/train/*/* > train.
scp
bfuentes@habla: /proyecto_hc/modelos/$ HCompV -C ../config/config -f 0.01 -
m -S train.scp -M hmm0 proto
```

Luego la informacion del piso de varianza queda en macros.

```
bfuentes@habla: /proyecto_hc/modelos/$ ../scripts/go.gen-macros ../hmm0/
vFloors hmm0/proto > hmm0/macros
cp /home/cestien/Documentos/go.gen-macros .
cp /home/cestien/Documentos/go.gen-hmmdefs ./
chmod + x go.gen-macros
chmod + x go.gen-hmmdefs

bfuentes@habla: /proyecto_hc/modelos/$ ../scripts/go.gen-hmmdefs ../
etc/monophones + sil
bfuentes@habla: /proyecto_hc/modelos/$ hmm0/proto > hmm0/hmmfeds
bfuentes@habla: /proyecto_hc/modelos/$ cat ../scripts/go.gen-macros
../scripts/go.gen-hmmdefs ../etc/monophones + sil > hmm0/macros
```

FINALMENTE, luego de haber creado los modelos iniciales estos se deben recalculan usando HEst.

HEst es la herramienta de HTK que reestima (entrena) los parámetros de los HMM usando el algoritmo Baum–Welch (EM).

Aque las etiquetas se usan para armar la topologia de la cadena de cada oración.

*Recordar que es no supervisado.*

```
bfuentes@habla: /proyecto_hc/modelos/$ mkdir hmm1 hmm2 hmm3
es la misma carpeta de modelos se hace:

HEst -T 1 -C ../config/config -I ../etc/fonemas0.mlf -t 250.0
150.0 1000.0 -S train.scp -H hmm0/macros -H hmm0/hmmfeds -M hmm1
../etc/monophones+sil
```

```

HERest -T 1 -C ../config/config -I ../etc/fonemas0.mlf -t 250.0
      150.0 1000.0 -S train.scp -H hmm1/macros -H hmm1/hmmdefs -M hmm2
      ../etc/monophones+sil

HERest -T 1 -C ../config/config -I ../etc/fonemas0.mlf -t 250.0
      150.0 1000.0 -S train.scp -H hmm2/macros -H hmm2/hmmdefs -M hmm3
      ../etc/monophones+sil

```

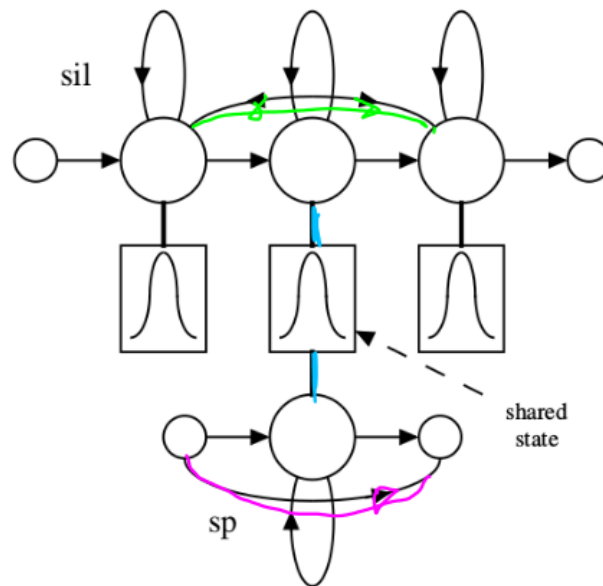
- **-C config**: archivo de configuración (parámetros de extracción y modelado).
- **-I fonemas0.mlf**: archivo MLF con las transcripciones de entrenamiento.
- **-S train.scp**: lista de archivos de características (MFCC).
- **-H macros**: carga parámetros globales del modelo (vFloors).
- **-H hmmdefs**: carga las definiciones de los HMM.
- **-M hmm<sub>i</sub>** : *directoriodesalidadelosmodelosreestimados*.
- **-t b1 b2 b3**: umbrales de poda para el algoritmo forward{backward}.
- **monophones+sil**: lista de modelos a entrenar.

La opción **-t** permite "podar" aquellos términos de las sumas en el algoritmo de forward-backward cuyo likelihood está debajo de un umbral. Dicho umbral se va incrementado si la reestimación da un error hasta un límite superior especificado en la opción.

## Modelo para sp

Una vez que se tiene el modelo reestimado, se agrega el modelo **sp** y se completa la topología del modelo **sil**. El modelo **sp** solo tiene un estado emisor. Se crea un prototipo con esas características en el **hmmdefs**. Usamos el estado central de **sil** y lo copiamos en el estado central de **sp**.

Faltaría agregar los arcos que completan la topología. Esto se hace con la herramienta HHed. HHed es un editor de modelos que se utiliza para definir modelos complicados a partir de modelos simples. En nuestro caso solo lo usamos para crear los arcos y para decirle a **sp** que comparte su estado central con el estado central de **sil**.



**Figura 2.4.3.2:** se agrega modelo de silencio

```
bfuentes@habla: /proyecto_hc/modelos/$ mkdir hmm4
bfuentes@habla: /proyecto_hc/modelos/$ cp hmm3/* hmm4/
```

Con sftp me descargo el archivo monophones+sil y con mi editor busco "sil" y aparece ~h "sil" y aparece otro sil igual, todo es bloque lo copio y modifico inmediatamente abajo.

Lo llamo ~h "sp" hago los ajustes necesarios para modelar sp y vuelvo a subir con el nombre monophones+sil+sp.

```
bfuentes@habla: /proyecto_hc/modelos/$ cp /home/cestine/Documentos/sil.hed
./
```

```
bfuentes@habla: /proyecto_hc/modelos/$ HHEd -T 1 -H hmm4/macros -H hmm4/
hmmdefs -M hmm5 sil.hed ../etc/monophones+sil+sp
```

Ahora, igual que antes uso HLEd para obtener los fonemas, ahora contemplando /sp/ por lo que se usa mkphones1.led.

Luego puede volver a re-estimar (entrenar) los modelos, ahora contemplando el modelo de /sp/

```
bfuentes@habla: /proyecto_hc/modelos/$ HLEd -l '*' -d dictl40 -i phones1.
mlf mkphones1.led mlftrain
```

```
bfuentes@habla: /proyecto_hc/modelos/$ mkdir hmm6 hmm7
```

```
bfuentes@habla: /proyecto_hc/modelos/$ HERest -T 1 -C ../config/config -I
../etc/phones1.mlf -t 250.0 150.0 1000.0 -S train.scp -H hmm5/
macros -H hmm5/hmmdefs -M hmm6 ../etc/monophones+sil+sp
```

```
bfuentes@habla: /proyecto_hc/modelos/$ HERest -T 1 -C ../config/config -I
../etc/phones1.mlf -t 250.0 150.0 1000.0 -S train.scp -H hmm6/
```

```
macros -H hmm6/hmmdefs -M hmm7 ../etc/monophones+sil+sp
```

#### 2.4.4. Reconocimiento con el modelo de unigramas y una sola gaussiana

En primer lugar deberá construirse el lattice (wdnet). Se implementaran dos tipos de redes, la primera utilizará todas las palabras del vocabulario con transiciones basadas en el modelo de lenguaje.

La segunda se implementará con una gramática simple (ver mas adelante).

El reconocimiento se implementa usando HVite. Las entradas serán, la lista de archivos mfc, el diccionario, la lista de monofonos (aqui debería incluirse el monofono sp), y el modelo. El resultado es un archivo que contiene las transcripciones por palabras reconocidas y las muestras en las que las mismas se encuentran.

La evaluación se realiza con HResults. Para ello es necesario la transcripción de los archivos de test a nivel palabra en un MLF.

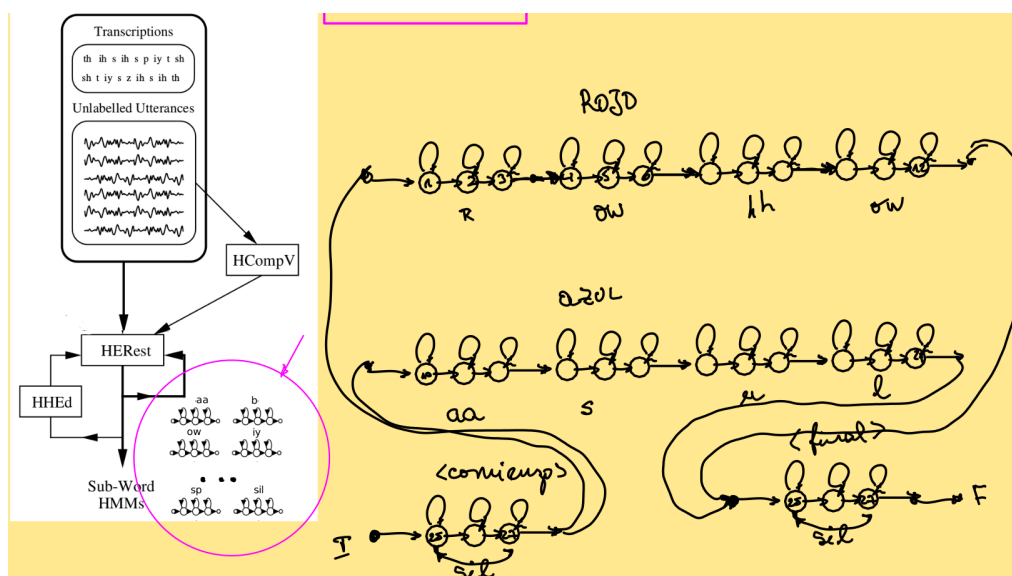


Figura 2.4.4.1: Esquema flujo de entrenamiento

EL proceso central utiliza el entrenamiento embebido **HREst** que utiliza enunciados de habla continua como fuente de datos de entrenamiento y re-estima simultáneamente el conjunto completo de HMMs de sub-palabras

”HREst necesita una transcripción, es decir, una lista de los fonemas presentes en dicho enunciado. A continuación, HREst concatena todos los HMMs de sub-palabras correspondientes a esa lista de fonemas para formar un único HMM compuesto. Este HMM compuesto se utiliza para recolectar las estadísticas necesarias para la re-estimación. Una vez procesados todos los enunciados de entrenamiento, el conjunto total de estadísticas acumuladas se emplea para re-estimar los parámetros de todos los HMMs de fonemas. Es importante enfatizar que, en este proceso, las transcripciones solo se necesitan para identificar la secuencia de fonemas en cada enunciado. No se requiere información sobre los límites temporales de los fonemas”

Un procedimiento de inicialización simple utiliza **HCompV** para asignar la media y la varianza globales del habla a cada distribución Gaussiana en cada HMM de fonema. Este

procedimiento, conocido como *flat start*, implica que durante el primer ciclo de re-estimación embebida, cada enunciado de entrenamiento será segmentado de manera uniforme. La expectativa es que una cantidad suficiente de los modelos de fonemas se alineen con realizaciones reales de ese fonema, de modo que en la segunda y posteriores iteraciones los modelos se alineen según lo esperado.

Se procede a generar los archivos pedidos.

```
/usr/local/speechapp/srilm/bin/i686-m64/ngram-count -order 2 -text
train.txt -lm lml40 -unkdiscount2 -vocab vocab

awk '{for(n=2;n<=NF;n++) print $n}' ../etc/prompts140.test |sort -u >
vocab
awk '{for(n=2;n<=NF;n++) printf "%s ",$n; print"\n"}' ../etc/
prompts140.train > train.txt

echo "<s>">> vocab
echo "</s>">> vocab
```

Se implementa un modelo de bigramas usando la técnica de conteo de palabras con la herramienta ngram-count.

- -order: máximo orden de ngram
- -text: archivo de texto a leer.
- -lm: LM a estimar
- -unkdiscount2: 2gram el descuento de Kneser-Ney
- -vocab: archivo de vocabulario.

El vocabulario consta de todas las palabras de la base de datos de test.

Por lo tanto usando prompts140.test se obtiene el vocabulario del mismo modo que obtuvo wlist140. El archivo train.txt contiene las transcripciones sin los identificadores.

El parámetro -order 2 indica que se implementa un modelo de bigramas y el parámetro unkdiscount2 indica que se usa el método de suavizado de Kneser-Ney.

Para completar el vocabulario se agrega la lista de líneas < s > y < /s >.

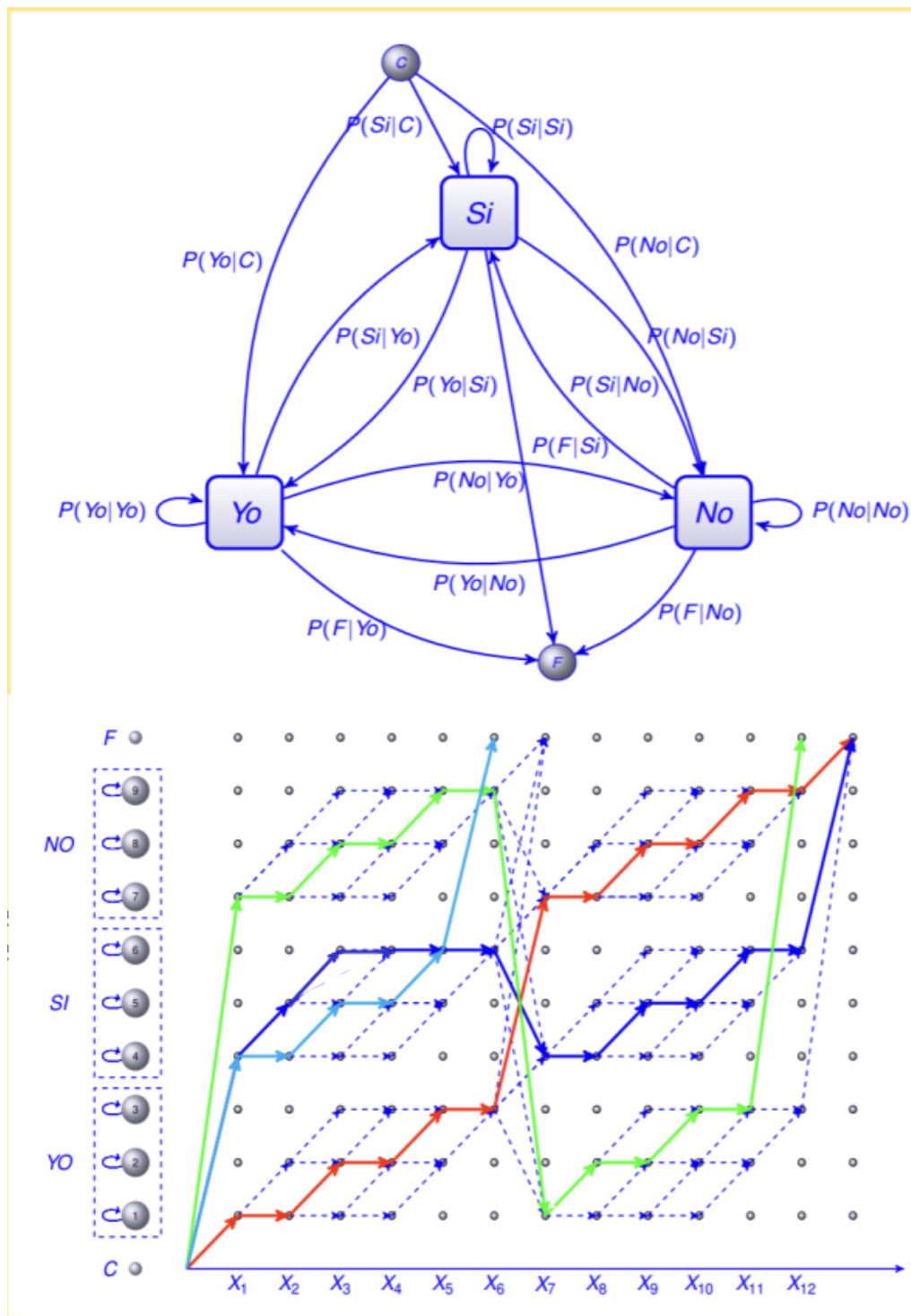
Para completar el diccionario se agrega al mismo las líneas < s > sil y < /s > sil

Luego se pasa lml40 al formato de HTK

```
HBUILD -n lml40 -s '<s>' '</s>' vocab wnet
```

## Testeo

En este inciso se busca realizar el reconocimiento automático del habla y verificar los resultados obtenidos a partir de los modelos entrenados en las secciones anteriores. Para ello, en primer lugar se genera el archivo **test.scp**, el cual contiene la lista de todos los archivos de audio que conforman el conjunto de prueba y que serán utilizados durante la etapa de reconocimiento.



**Figura 2.4.4.2:** Secuencia de viterbi y probabilidades de transición

```
ls ../datos/mfc/test/*/* > test.scp
```

A continuación, el proceso de reconocimiento se lleva a cabo mediante la herramienta **HVite**, la cual implementa el algoritmo de Viterbi para encontrar la secuencia de estados más probable dado un modelo acústico y una secuencia de observaciones. En una primera instancia, se aplica HVite sobre el modelo HMM7 (una gaussiana por estado), con el objetivo de evaluar su desempeño base.

Durante la decodificación, HVite combina la información del modelo acústico (probabilidades de emisión  $b_j(o_t)$ ) con las probabilidades de transición entre estados, construyendo una red de búsqueda que permite determinar la secuencia óptima de estados. A partir de dicha secuencia, se obtiene finalmente la secuencia de palabras reconocidas, respetando las restricciones impuestas por el modelo del lenguaje utilizado.

```
HVite -A -T 1 -C ../config/config -H ../modelos/hmm7/macros -H ../
      modelos/hmm7/hmmdefs -l '*' -i recout -w ../lm/wdnet -S test.scp
      -p 0.0 -s 5.0 ../etc/dictl40 ../etc/monophones+sil+sp
```

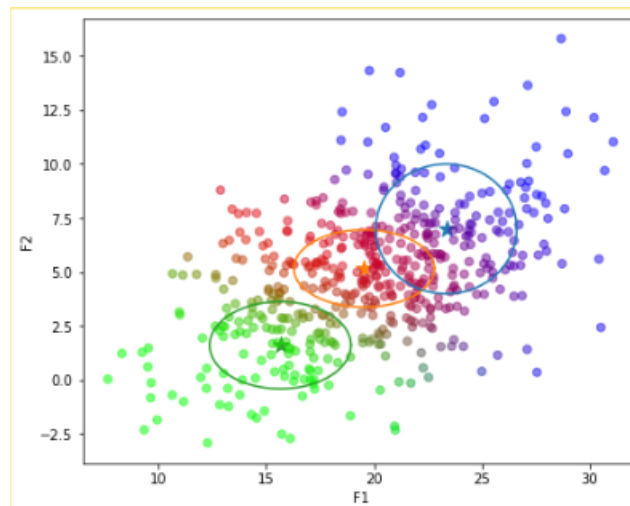
- -C: Especifica el archivo de configuración
- -H: Indica los archivos del modelo acústico (macros y definiciones de HMM)
- -S: Define el archivo que contiene la lista de los archivos de audio de test.
- -i: Especifica el archivo de salida en formato MLF, donde se almacenan las transcripciones resultantes del reconocimiento.
- -w: Indica el archivo que describe la red gramatical o el modelo del lenguaje, el cual define las probabilidades de transición entre palabras 2.4.4.2.
- -p: Configura la penalización por inserción de palabras, utilizada para controlar la aparición de palabras espurias durante el reconocimiento.
- -s: Define el factor de escala gramatical, el cual ajusta la influencia relativa del modelo del lenguaje respecto del modelo acústico.

Luego se muestran los resultados del testeo

```
HResults -t -f -I ../etc/mlftest ../etc/monophones+sil+sp recout2g
```

y este comando nos dá:

```
vm25_125.rec: 33.33( 22.22) [H= 3, D= 2, S= 4, I= 1, N= 9]
Aligned transcription: vm25_125.lab vs vm25_125.rec
LAB:      monselas nego   que le hubieran pagado por el   homicidio
REC: sus el      espero que   hubiera pagado      cual homicidio
----- Overall Results -----
SENT: %Correct=7.50 [H=74, S=913, N=987]
WORD: %Corr=59.95, Acc=47.07 [H=4742, D=415, S=2753, I=1019, N=7910]
=====
```



**Figura 2.4.4.3:** mezcla de 3 gaussianas

Donde se espera que los labels (LAB) y lo recuperado (REC) sean lo mismo. **H** son las palabras correctas recuperadas y **N** son las totales, **D** son las borradas, **S** son las que sustituyó e **I** las inserciones; en este ejemplo insertó “sus”.  
y así se definen las siguientes métricas.

$$\begin{aligned} \%Correct &= \frac{H}{N} \times 100 \\ \%Accuracy &= \frac{H - I}{N} \times 100 \end{aligned} \quad (2.1)$$

Ahora vemos lo siguiente, da horrible, muy mal. Bueno, esto se puede mejorar aumentando la cantidad de gaussianas, dado que se está entrenando con una sola gaussiana que encima es diagonal. La cantidad de gaussianas se puede aumentar y se pasaría a modelar con una mezcla de gaussianas, por ejemplo.

#### 2.4.5. Refinamiento del modelo agregando mezclas de gaussianas

Se procederá a refinar el modelo partiendo del modelo de una gaussiana. Se creará un modelo de dos mezclas usando **HHEd** y se estimará dicho modelo usando **HERest**. Luego se evaluará el desempeño de dicho modelo y se continuará incrementando el número de gaussianas en potencias de dos, hasta que el incremento del desempeño sea despreciable.

Ok, vamos a hacerlo: en primer lugar tengo una gaussiana por estado y con **HHEd** voy a llevarla a 2 gaussianas por estado, y así voy a ir incrementando en potencias de 2 hasta que mis métricas no varíen demasiado.

Para ello se debe tener un archivo que modifique los modelos; este será copiado.

```
bfuentes@habla: /proyecto_hc/modelos\$ cat editf2g
MU 2 {*.state[2-5].mix}
```

**MU m itemList(p)**

MU 2: para poner 2 mezclas y luego sigue el argumento de a quienes aplicar

\*.state[2-5].mix: a todas las unidades (\*) a todos los estados entre 2 y 5(.state[2-5]) y a todas las mezclas (.mix)



vamos a considerar que todos los estados de todas las subunidades van a tener siempre la misma cantidad de mezclas de gaussianas y nuestro objetivo es ver hasta qué número de mezclas de gaussianas vamos a incrementar

Finalmente lo que se hace es lo siguiente

```
cd ./modelos

HHEd -T 1 -H hmm7/macros -H hmm7/hmmdefs -M hmm2g.0/ editf2g ../etc
/monophones+sil+sp

HERest -A -T 2 -C ../config/config -I ../etc/phones1.mlf -t 250.0
150.0 1000.0 -S train.scp -H hmm2g.0/macros -H hmm2g.0/hmmdefs -M
hmm2g.1 ../etc/monophones+sil+sp

HERest -A -T 2 -C ../config/config -I ../etc/phones1.mlf -t 250.0
150.0 1000.0 -S train.scp -H hmm2g.2/macros -H hmm2g.2/hmmdefs -M
hmm2g.3 ../etc/monophones+sil+sp

cd ../rec/

HVite -A -T 1 -C ../config/config -H ../modelos/hmm2g.2/macros -H
../modelos/hmm2g.2/hmmdefs -l '*' -i recout2g -w ../lm/wdnet -S
test.scp -p 0.0 -s 5.0 ../etc/dictl40 ../etc/monophones+sil+sp &

HResults -t -f -I ../etc/mlftest ../etc/monophones+sil+sp recout2g
```

Esto dio los siguientes resultados

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest ../etc/monophones+si
...
----- Overall Results -----
SENT: %Correct=11.75 [H=116, S=871, N=987]
WORD: %Corr=64.64, Acc=53.43 [H=5113, D=370, S=2427, I=887, N=7910]
=====
```

amos que mejoró, y lo que se hizo fue editar con HHEd para tener 2 gaussianas; eso quedó guardado en `hmm2g.0` y posteriormente se le dio 2 pasadas de entrenamiento con HERest, las cuales fueron guardadas en `hmm2g.1` y `hmm2g.2` respectivamente.

Finalmente se testea el último entrenamiento con HVite y su resultado queda en `recout2g`, el cual se mostrará con HResult.

Estos mismos pasos van a ser repetidos y, en total, todos los resultados —incluidos los anteriores— se muestran a continuación.

**ATENCIÓN:** En una primera aproximación no cambié el archivo de configuración `edit2g`; esto evitó que mejorara mucho más del 70 %. Por ello generé todos los archivos de configuración a la vez y reentreno desde 4 gaussianas.

Se generan todos los archivos de configuración:

```
bfuentes@habla: ~/proyecto_hc/modelos/$
for i in {2..8}; do n=$((2*i)); echo "MU $n {*.state[2-5].mix}" > "
editf${n}g"; done
```

y los resultados son:

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout
...
----- Overall Results -----
SENT: %Correct=7.50 [H=74, S=913, N=987]
WORD: %Corr=59.95, Acc=47.07 [H=4742, D=415, S=2753, I=1019, N=7910]
=====
```

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout2g
...
----- Overall Results -----
SENT: %Correct=11.75 [H=116, S=871, N=987]
WORD: %Corr=64.64, Acc=53.43 [H=5113, D=370, S=2427, I=887, N=7910]
=====
```

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout4g
...
----- Overall Results -----
SENT: %Correct=20.77 [H=205, S=782, N=987]
WORD: %Corr=71.59, Acc=63.30 [H=5663, D=323, S=1924, I=656, N=7910]
=====
```

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout8g
...
----- Overall Results -----
SENT: %Correct=29.58 [H=292, S=695, N=987]
WORD: %Corr=77.55, Acc=70.96 [H=6134, D=271, S=1505, I=521, N=7910]
=====
```

```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout16g
...
----- Overall Results -----
SENT: %Correct=35.36 [H=349, S=638, N=987]
WORD: %Corr=80.57, Acc=75.18 [H=6373, D=263, S=1274, I=426, N=7910]
=====
```

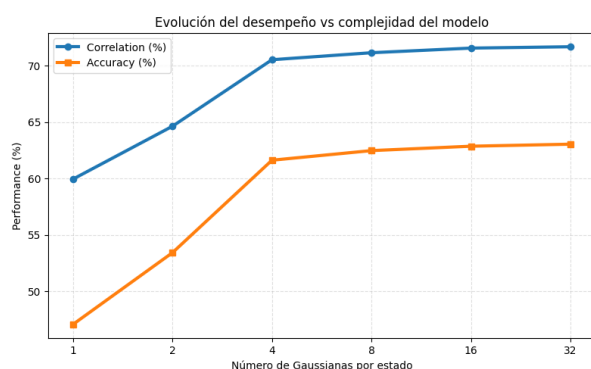
```
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout32g
...
----- Overall Results -----
SENT: %Correct=40.73 [H=402, S=585, N=987]
WORD: %Corr=82.81, Acc=78.43 [H=6550, D=230, S=1130, I=346, N=7910]
=====

bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout64g
...
----- Overall Results -----
SENT: %Correct=44.17 [H=436, S=551, N=987]
WORD: %Corr=84.58, Acc=80.57 [H=6690, D=197, S=1023, I=317, N=7910]
=====

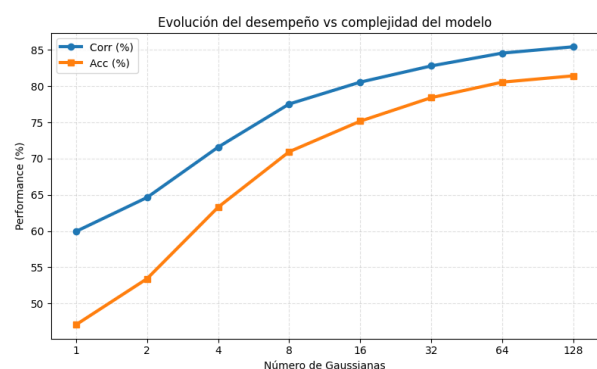
bfuentes@habla:~/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest
../etc/monophones+sil+sp recout128g
...
----- Overall Results -----
SENT: %Correct=48.36 [H=476, S=521, N=987]
WORD: %Corr=86.61, Acc=82.92 [H=6820, D=166, S=916, I=266, N=7910]
=====
```

Vemos que se llegó hasta 128 mezclas de gaussianas y, si se grafican las métricas, estas rondan el 85 % de \*correctness\* 2.4.5.1b, con una variación de aproximadamente 1 % en las métricas. Seguir aumentando la complejidad no va a mejorar mucho más las métricas.

Adicionalmente, se grafica en la 2.4.5.1a las métricas con dos gaussianas (por error, pero sirve para la comparación); en ese caso, si seguíamos entrenando unas pasadas más, podríamos mejorar bastante, pero rápidamente queda en un \*correctness\* que ronda el 70 %. Esto refuerza la idea de que se debía aumentar la complejidad.



(a) Métricas(es siempre 2g, pero mas entrenamiento)



(b) Métrica en potencia de 2g

**Figura 2.4.5.1:** Comparación de métricas

Haciendo el calculo de la variación porcentual de las respectivas metricas se tiene

- $\Delta$  Accuracy última iteración: 1.080 %
- $\Delta$  Correlation última iteración: 1.029 %

Con lo cual se concluye que ya fueron mezclas suficientes, a partir 64 0 128 gaussianas ya no es necesario agregar mas complejidad.

## 2.5. Implementación de la gramática finita.

La tarea consiste en un sistema de reconocimiento de llamadas telefónicas similar al descrito en el tutorial de HTK. En nuestro caso, se definen los nodos **env-com** y **env-fin**, que representan el comienzo y fin de frase. Seguidamente a **env-com** pueden ocurrir las siguientes frases:

1. “llame” / “llamar” “al” seguido de los digitos 0 a 9 dichos cualquier cantidad de veces.
2. “comuniqueme con” seguido de los nombres juan, juana, patricia, pedro, andrea y andres. Opcionalmente los nombres juan pedro y andrea pueden venir acompañados de sus respectivos apellidos: fernandez, rodriguez y perez.

Los archivos *wlist.gf* y *lexicon.gf*, en el directorio del proyecto, contienen la lista de palabras de la gramática y el diccionario de pronunciaciones de la misma, respectivamente.

```
bfuentes@habla: /proyecto_hc/etc/$ cp /home/cestien/Documentos/wlist.gf .
```

### HParse

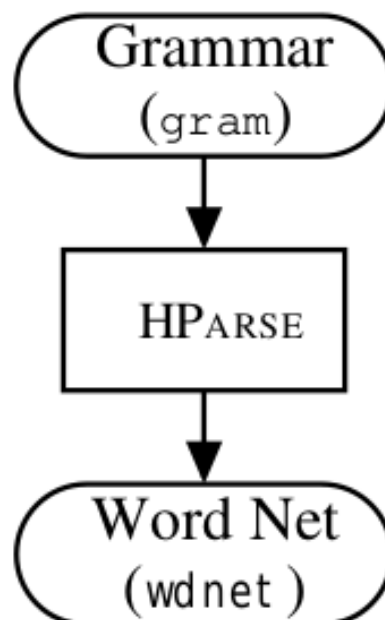


Figura 2.5.1: HParse

HTK proporciona un lenguaje de definición de gramáticas para especificar gramáticas de tareas simples como esta. Dicho lenguaje consiste en un conjunto de definiciones de variables, seguido por una expresión regular que describe las palabras a reconocer.

Para la aplicación de marcado por voz, una gramática adecuada podría ser la siguiente:

```
$digit = cero | uno | dos | tres | cuatro | cinco | seis | siete | ocho | nueve ;

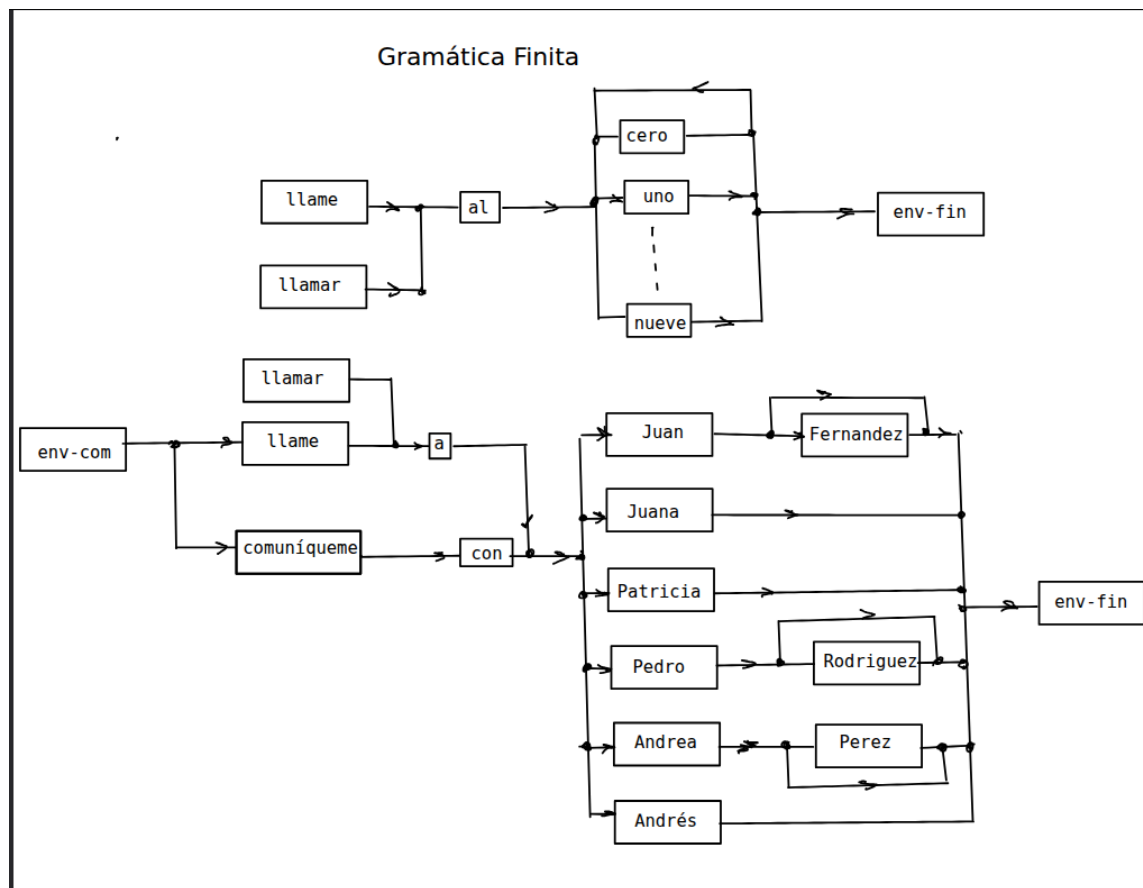
$name = juan [fernandez]
      | juana
      | patricia
      | pedro [rodriguez]
      | andrea [perez]
      | andres ;

( env-com
(
  (llame | llamar) al <$digit>
|
( (llamar|llame) a | comuniqueme con ) <$name>
)
env-fin
)
```

En esta notación,

- " | ": Indican alternativas.
- " [ ] ": Indican elementos opcionales.
- " < > ": Indican una o mas repeticiones.
- " ( ) ": Se usa para agrupar sub-expresiones

La gramática completa puede representarse como una red, tal como se muestra en la figura correspondiente.



**Figura 2.5.2:** Gramatica finita

El reconocedor de HTK requiere que la gramática sea definida mediante una red de palabras utilizando una notación de bajo nivel denominada *HTK Standard Lattice Format* (SLF), en la cual cada instancia de palabra y cada transición palabra-a-palabra se encuentran explícitamente especificadas.

Esta red de palabras puede generarse de forma automática a partir de la gramática de alto nivel utilizando la herramienta **HParse**, fig: 2.5.1.

De este modo, asumiendo que el archivo **gram** contiene la gramática definida anteriormente, la ejecución del comando

```
HParse gram wdnnet
```

generará una red de palabras equivalente en el archivo **wdnnet** que se llamará **wdnnet\_gf**.

```
/proyecto_hc/etc/$ nano gram          (se copia la gramatica finita)
/proyecto_hc/lm/$  HParse ../etc/gram wdnnet_gf
```

```
bfuentes@habla:~/proyecto_hc/lm$ head wdnnet_gf
VERSION=1.0
N=36   L=66
I=0     W=env-fin
I=1     W=andres
```

```

I=2    W=NULL
I=3    W=perez
I=4    W=andrea
I=5    W=rodriguez
I=6    W=pedro
I=7    W=patricia

```

### Generacion de frase aleatorias

*A fin de evaluar los modelos fonéticos obtenidos anteriormente con la tarea de gramática finita, se deben generar 200 frases aleatorias usando HSGen.*

Primero hay que generar los diccionarios de palabras a fonemas; para ello habrá que usar **HDMan**, que unificará distintas fuentes para generar *dict\_gf*.

Si bien *lexicon.gf* no tiene “a”, esto no importa dado que *lexicon.gf* se usa para complementar **lexicon**, el cual sí lo tiene.

```

bfuentes@habla:~/proyecto_hc/etc$ head lexicon
a                                aa
abaco                           aa bb aa k ow
...

```

```

bfuentes@habla: ~/proyecto_hc/etc/$ HDMan -m -w wlist.gf -g global.ded -n
monophones+sil -l ../log/hdman_gf.log dict_gf lexicon lexicon.gf

```

A diferencia del lo hecho antes, acá cambia **wlist** por **wlist.gf** y el resultado es el diccionario **dict\_gf**

```

bfuentes@habla:~/proyecto_hc/etc$ cat dict_gf
a                aa sp
al              aa l sp
andrea          aa n d rx ey aa sp
andres          aa n d rx ey s sp
cero            s ey rx ow sp
cinco           s iy n k ow sp
comuniqueme     k ow m uw n iy k ey m ey sp
con             k ow n sp
cuatro          k w aa t rx ow sp
dos             d ow s sp
fernandez       f ey rx n aa n d ey s sp
juan            hh w aa n sp
juana           hh w aa n aa sp
llamar          y aa m aa rx sp
llame           y aa m ey sp
nueve           n w ey bb ey sp
ocho            ow ch ow sp
patricia        p aa t rx iy s y aa sp
pedro           p ey dd rx ow sp
perez           p ey rx ey s sp

```

```

rodriguez      rr ow dd rx iy gg ey s sp
seis           s ey y s sp
siete         s y ey t ey sp
tres          t rx ey s sp
uno           uw n ow sp

```

Ahora vemos que faltan las entradas **env-com** y **env-fin**; se las agrega manualmente con un modelo de silencio (como muestra el libro de HTK).

```

env-com [] sil
env-fin [] sil

```

Ahora si podemos usar **HSGen**

```

bfuentes@habla: /proyecto_hc/etc\$
bfuentes@habla:~/proyecto_hc/etc$ HSGen -l -n 200 ../lm/wdnet_gf
dict_gf > prompts_gf.test

```

El cual genera las 200 palabras pedidas; estas serán grabadas para su posterior testeo con el mejor modelo obtenido.

```

bfuentes@habla:~/proyecto_hc/etc$ head prompts_gf.test
1. llame a juana
2. comuniqueme con patricia
3. llamar a patricia
4. llame a andrea perez
5. llame al tres
6. llamar a andres
7. llame al dos siete dos ocho ocho cuatro tres cero seis
8. llamar a juan fernandez
9. llame a andres
10. llame al nueve nueve ocho cuatro dos uno tres tres cero tres cuatro cero tres c

bfuentes@habla:~/proyecto_hc/etc$ tail prompts_gf.test
191. llamar a andrea perez
192. llamar al ocho tres tres cuatro ocho siete ocho cuatro cinco seis cero cinco n
193. llamar a andrea
194. llamar al cuatro cero cuatro dos cero ocho seis ocho cuatro tres seis dos dos
195. llame a andrea
196. llamar al cero
197. llamar a juana
198. comuniqueme con juan fernandez
199. llamar a juana
200. llame a juana

```

Notar que son frases que tienen sentido según el diagrama 2.5.2; este representa un sistema de llamadas automáticas con detección de voz.

A continuación, se procede a grabar todos estos ejemplos generados aleatoriamente con mi propia voz.



Muy bien, solo queda generar los archivos mfc para comenzar; para ello se utiliza nuevamente **HCopy**, como se mostró en 2.3.1.

**HCopy** recibe el archivo de configuración *hconfig\_gf.hcopy*, que contiene toda la configuración —la misma que *hconfig.copy*— y una lista **genmfc\_gf.test** que contiene las rutas de los archivos de entrada y la ruta de salida. Esta debe ser generada con *go\_gf.mfclist* (modificando *go.mfclist*).

**Nota:** *El archivo de configuración en ambos casos es el mismo salvo por una diferencia: en el archivo original, el tipo de entrada figura como NIST, mientras que en nuestro caso es WAV. Sin embargo, revisando los datos tienen extensión \*.wav, aunque el archivo de entrada está declarado como NIST. En nuestro caso es WAV, como se puede ver en el siguiente comando.*

**Nota:** *Hubo que resamplear los audios para que coincidiera la frecuencia de muestreo; el código se incluye en el anexo.*

```
bfuentes@habla:~/proyecto_hc/datos$ HList -F WAV -h -e 10 wav_gf/test/brian/001.wav
----- Source: wav_gf/test/brian/001.wav -----
Sample Bytes: 2          Sample Kind:  WAVEFORM
Num Comps:    1          Sample Period: 62.5 us
Num Samples:  43415      File Format:   WAV
----- Samples: 0->11 -----
  0:   -1257  -2131  -1890  -2047  -1941  -2047  -1937  -2000  -1971  -2014
 10:   -1984  -2003
----- END -----
```

```
bfuentes@habla: ~/proyecto_hc/datos/$ HCopy -T 1 -C ../config/config_gf.
hcopy -S genmfc_gf.test
```

Luego produce archivos HTK, dejándonos en el mismo formato que los datos de entrenamiento.

```
~/proyecto_hc/datos$ HList -F HTK -h -e 10 mfc_gf/test/brian/001.mfc
----- Source: mfc_gf/test/brian/001.mfc -----
Sample Bytes: 26          Sample Kind:  MFCC_C_K_0
Num Comps:    13          Sample Period: 10000.0 us
Num Samples:  269         File Format:   HTK
----- Samples: 0->10 -----
 0:   -13.014  4.790  0.025  6.043  4.830  5.436  4.882  7.475  4.893  7.
      4.885  6.476 64.450
 1:   -12.834  5.510  2.406  6.684  2.558  7.557  7.553  6.334  5.320  6.
      6.842  4.903 64.025
 2:   -12.879  5.065  2.838  7.310  3.925  6.373  6.668  3.749  5.921  9.
      7.507 11.221 64.273
 3:   -13.028  3.759  0.209  4.769  3.156  5.616  7.802  5.500  1.930  7.
      7.232  7.143 64.117
 4:   -12.632  3.788  0.774  5.411  6.198 12.238  8.347  5.043  4.086  6.
      4.823  3.557 64.322
```

|     |         |       |        |       |        |        |        |        |       |     |
|-----|---------|-------|--------|-------|--------|--------|--------|--------|-------|-----|
| 5:  | -13.116 | 5.459 | 0.881  | 4.001 | 1.480  | 5.704  | 6.194  | 6.951  | 6.993 | 6.  |
|     | 5.762   | 3.824 | 64.164 |       |        |        |        |        |       |     |
| 6:  | -13.579 | 6.637 | 5.318  | 7.804 | 3.546  | 6.470  | 6.636  | 5.558  | 4.986 | 7.  |
|     | 6.068   | 5.016 | 63.749 |       |        |        |        |        |       |     |
| 7:  | -13.987 | 3.482 | 1.840  | 6.376 | 3.900  | 8.408  | 11.505 | 9.908  | 5.915 | 6.  |
|     | 5.561   | 5.093 | 63.701 |       |        |        |        |        |       |     |
| 8:  | -13.671 | 4.079 | 1.854  | 7.403 | 6.792  | 10.350 | 9.988  | 8.699  | 6.326 | 6.  |
|     | 1.107   | 1.823 | 64.203 |       |        |        |        |        |       |     |
| 9:  | -13.671 | 3.168 | 3.962  | 6.284 | 3.203  | 9.033  | 7.673  | 5.048  | 3.019 | 8.  |
|     | 4.866   | 6.041 | 64.640 |       |        |        |        |        |       |     |
| 10: | -14.091 | 0.844 | 7.317  | 8.800 | -2.648 | 10.768 | 6.597  | -2.940 | 5.984 | 11. |
|     | 1.533   | 9.642 | 65.863 |       |        |        |        |        |       |     |

----- END -----

### Por que cambia tanto los ceptrum con la frecuencia de muestreo?

Los coeficientes cepstrales se calculan sobre un eje de frecuencia que depende directamente de la frecuencia de muestreo. Al modificarla, se reescala el eje de frecuencia, por lo que un mismo contenido espectral se proyecta en distintas bandas de frecuencia respecto del conjunto de entrenamiento, generando un desajuste de dominio.

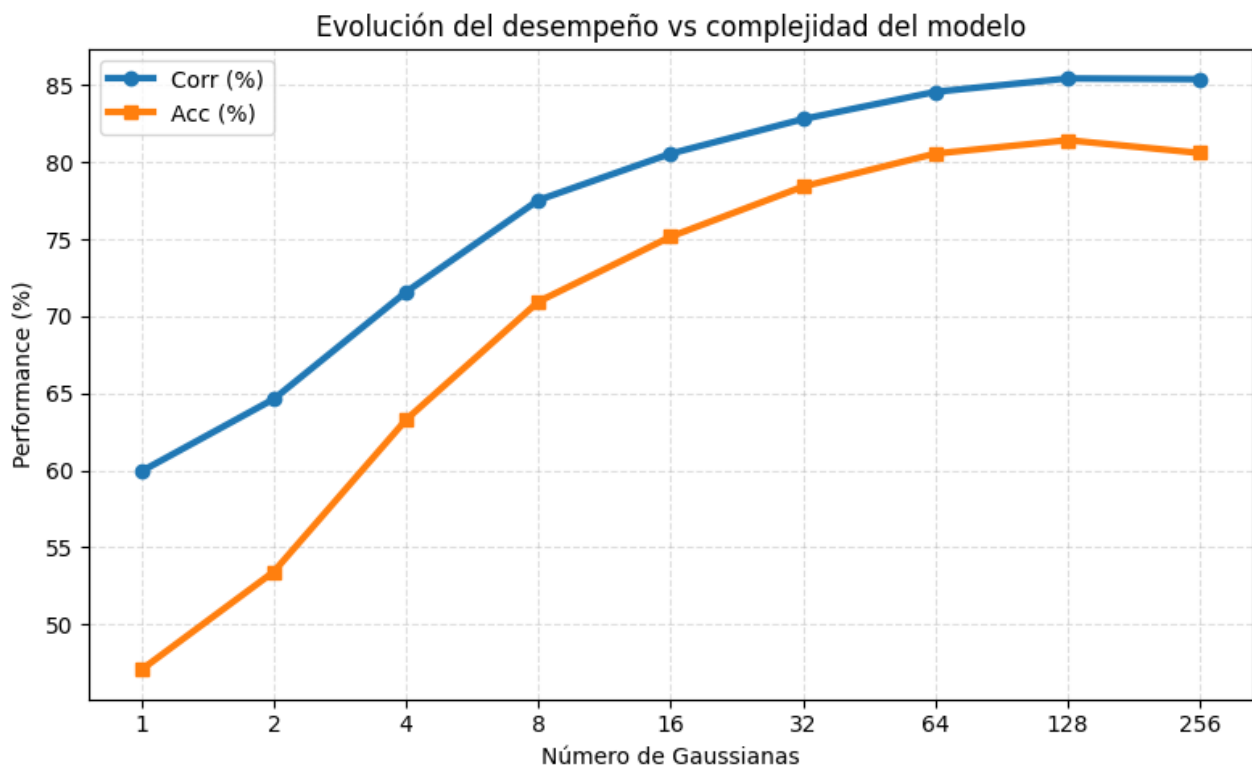
En particular, el banco de filtros Mel captura energía en bandas redefinidas por la nueva frecuencia de muestreo, produciendo vectores de coeficientes cepstrales donde su distribución estadística no es comparable con la utilizada durante el entrenamiento del modelo.

(Intuitivamente, si miramos la Fig. 2.4.4.3, podemos pensar que las muestras se desplazan hacia el interior, un reescalado, lo que provoca que el reconocimiento de los clusters sea erróneo.)

#### 2.5.1. Evaluación

Ahora se procede a evaluar el modelo del sistema de la misma manera que en la primera parte, excepto que la red **wnet** ahora es **wnet\_gf**.

Se realizará la evaluación con el mejor modelo obtenido. Para esto, se entrenó un modelo más complejo con 256 gaussianas, pero no superó al de 128g.



**Figura 2.5.1.1:** Mdelos entrenados

Primero tengo que generar el archivo **test\_gf.scp**, que contiene las rutas de los archivos \*.mfc generados, relativas a la ruta *../rec*.

Finalmente, se realiza la evaluación con el algoritmo Viterbi, usando **HVite**.

```
cd ../rec
ls ../datos/mfc_gf/test/brian/* > test_gf.scp

HVite -A -T 1 \
-C ../config/config \
-H ../modelos/hmm128g.2/macros \
-H ../modelos/hmm128g.2/hmmdefs \
-l '*' \
-i recout128g_gf \
-w ../lm/wdnet_gf \
-S test_gf.scp \
-p 0.0 \
-s 5.0 ../etc/dict_gf ../etc/monophones+sil+sp &
```

Finalmente, se observan los resultados.

No sin antes generar el archivo **mlftest\_gf** usando el script **prompts2mlf** y los prompts generados anteriormente **prompts<sub>gf</sub>.test**.

Lamentablemente, el archivo **prompts2mlf** no se generó usando tres cifras, como los tengo yo en mis archivos, por lo que causaría un error de nombres más adelante. Lo soluciono cambiando los nombres de la siguiente manera.

```

proyecto/etc
cp prompts_gf.test prompts_gf.testBKP
sed 's/^[0-9]\+\.\s*/' prompts_gf.testBKP | nl -w3 -nrz -s'. ' >prompts_gf.test

```

Tambien como **prompts2mlf** generaba el nombre con un punto de mas se le agrego la siguiente linea

```
$fname =~ s/\.$//
```

Luego se guardo los cambios en **prompts2mlf\_gf**.

Ahora si....

```
/proyecto_hc/etc$ ../scripts/prompts2mlf mlftest_gf prompts_gf.test
```

```
/proyecto_hc/rec$ HResults -t -f -I ../etc/mlftest_gf ../etc/monophones+sil+sp reco
```

Aligned transcription: 194.lab vs 194.rec

```

LAB: llamar al cuatro cero      cuatro dos cero      ocho seis      ocho cu
REC: llamar al cuatro cero dos cuatro dos cero uno uno cero seis dos cero nueve ce
195.rec: 100.00(100.00) [H= 3, D= 0, S= 0, I= 0, N= 3]
196.rec: 100.00(100.00) [H= 3, D= 0, S= 0, I= 0, N= 3]
197.rec: 100.00(100.00) [H= 3, D= 0, S= 0, I= 0, N= 3]
198.rec: 100.00(100.00) [H= 4, D= 0, S= 0, I= 0, N= 4]
199.rec: 100.00(100.00) [H= 3, D= 0, S= 0, I= 0, N= 3]
200.rec: 100.00(100.00) [H= 3, D= 0, S= 0, I= 0, N= 3]

```

```

----- Overall Results -----
SENT: %Correct=62.76 [H=123, S=73, N=196]
WORD: %Corr=90.19, Acc=69.88 [H=1443, D=6, S=151, I=325, N=1600]
=====

```

Vemos que, efectivamente, el modelo funciona para nuevas palabras. Además, es de recalcar que las primeras 100 frases fueron generadas en tiempos distintos, por lo que la distribución y el nivel de ruido cambiaron, y aun así el modelo fue lo suficientemente robusto.

### 3. Conclusiones

Se concluye que es posible realizar el entrenamiento con solo 2 gaussianas, obteniendo una corrección cercana al 70 %. Si bien este resultado surgió de un error inicial, permitió observar que, con más iteraciones de entrenamiento, el modelo de 2 gaussianas aún tenía margen de mejora.

Por otro lado, es indudable que aumentar la complejidad del modelo mejora su desempeño. Incrementando el número de gaussianas en potencias de dos, se llegó hasta un modelo de 256 gaussianas, donde la mejora resultó prácticamente insignificante (incluso empeoro el acc). Así, el mejor rendimiento se obtuvo con el modelo de 128 gaussianas.

El modelo entrenado demostró ser robusto no solo con los datos utilizados en el entrenamiento, sino también frente a nuevas entradas de audio. Esto incluye variaciones en el ruido y en las condiciones de grabación, especialmente considerando que las primeras 100 frases se produjeron en momentos distintos.

Finalmente, es importante recalcar que el formato de los audios debe mantenerse tanto para el entrenamiento como para el testeo, a fin de asegurar coherencia y evitar incompatibilidades durante la evaluación.

## Referencias

- [1] Cambridge University Engineering Department (2006). *The HTK Book*.

## 4. ANEXO

### Script simple para cambiar la periodo de muestreo

```
import os
import librosa
import soundfile as sf

sr_deseado = 16000 #1/62.5 uSeg
os.makedirs('wav_16k', exist_ok=True)

for archivo in os.listdir('wav'):
    if archivo.endswith('.wav'):
        y, _ = librosa.load(f'wav/{archivo}', sr=sr_deseado)
        sf.write(f'wav_16k/{archivo}', y, sr_deseado)
```

### Modificación go.mfclist

```
cat ../scripts/go_gf.mfclist
#!/bin/sh
#Crea las listas archivos a procesar por HCopy
#y los directorios en mfc, Ejecutarlo desde datos
# igual que el original pero solo para test y con _gf

if [ $# -ne 1 ]
then
    echo Usar: $0 listatest
    exit 1
fi

ls wav_gf/test |\
while read file
do
    mkdir -p mfc_gf/test/$file
done

ls wav_gf/test/*/* >p
```

```
cat p \  
| sed 's/wav_gf/mfc_gf/g' \  
| sed 's/wav/mfc/g' \  
> q  
paste p q > $1  
rm p q
```